```
import java.util.*;

class RabbitLeapDFS {

    public List<String> initial() {
        return Arrays.asList("W", "W", "W", "_", "E", "E", "E");
    }

    public boolean isGoal(List<String> state) {
        return state.equals(Arrays.asList("E", "E", "E", "_", "W", "W", "W"));
    }

    public List<List<String>> nextMove(List<String> state) {
        List<List<String>> moves = new ArrayList<>();
        int n = state.size();

        for (int i = 0; i < n; i++) {
            String curr = state.get(i);

            if (curr.equals("W")) {
                // W moves right by 1
                if (i + 1 < n && state.get(i + 1).equals("_")) {
                    List<String> newState = new ArrayList<>(state);
                    Collections.swap(newState, i, i + 1);
                    moves.add(newState);
                }
                // W jumps right by 2
                else if (i + 2 < n && !state.get(i + 1).equals("_") && state.get(i +
2).equals("_")) {
                    List<String> newState = new ArrayList<>(state);
                    Collections.swap(newState, i, i + 2);
                    moves.add(newState);
                }
            }
            else if (curr.equals("E")) {
                // E moves left by 1
                if (i - 1 >= 0 && state.get(i - 1).equals("_")) {
                    List<String> newState = new ArrayList<>(state);
                    Collections.swap(newState, i, i - 1);
                    moves.add(newState);
                }
                // E jumps left by 2
```

```
            else if (i - 2 >= 0 && !state.get(i - 1).equals("_") && state.get(i -
2).equals("_")) {
                List<String> newState = new ArrayList<>(state);
                Collections.swap(newState, i, i - 2);
                moves.add(newState);
            }
        }
    }
    return moves;
  }
}

public class DFSRabbitLeapSolver {

  public static List<List<String>> dfs(RabbitLeapDFS problem) {
    Stack<Pair> stack = new Stack<>();
    stack.push(new Pair(problem.initial(), new ArrayList<>()));
    Set<List<String>> visited = new HashSet<>();

    while (!stack.isEmpty()) {
      Pair current = stack.pop();
      List<String> state = current.state;
      List<List<String>> path = current.path;

      if (problem.isGoal(state)) {
        path.add(state);
        return path;
      }

      if (visited.contains(state)) {
        continue;
      }

      visited.add(state);

      for (List<String> nextState : problem.nextMove(state)) {
        List<List<String>> newPath = new ArrayList<>(path);
        newPath.add(state);
        stack.push(new Pair(nextState, newPath));
      }
    }
    return null;
```

```
    }

    // Helper class to hold state and path
    static class Pair {
        List<String> state;
        List<List<String>> path;

        Pair(List<String> state, List<List<String>> path) {
            this.state = state;
            this.path = path;
        }
    }

    public static void main(String[] args) {
        RabbitLeapDFS dfsProblem = new RabbitLeapDFS();
        List<List<String>> dfsResult = dfs(dfsProblem);

        System.out.println("DFS Path:");
        for (List<String> step : dfsResult) {
            System.out.println(step);
        }
    }
}
```

```
import java.util.*;

class BridgeCrossingBFS {

    Map<String, Integer> times = Map.of(
        "Amogh", 5,
        "Ameya", 10,
        "Grandmother", 20,
        "Grandfather", 25
    );

    Set<String> people = new HashSet<>(times.keySet());

    // Initial State
    public State initial() {
```

```
        return new State(new HashSet<>(people), new HashSet<>(), "left", 0);
    }

    // Goal Test
    public boolean isGoal(State state) {
        return state.left.isEmpty() && state.time <= 60;
    }

    // Generate Next Moves
    public List<State> nextMove(State state) {
        List<State> moves = new ArrayList<>();

        if (state.side.equals("left")) {
            List<String> leftList = new ArrayList<>(state.left);

            for (int i = 0; i < leftList.size(); i++) {
                for (int j = i; j < leftList.size(); j++) {
                    String p1 = leftList.get(i);
                    String p2 = leftList.get(j);

                    Set<String> newLeft = new HashSet<>(state.left);
                    newLeft.remove(p1);
                    newLeft.remove(p2);

                    Set<String> newRight = new HashSet<>(state.right);
                    newRight.add(p1);
                    newRight.add(p2);

                    int newTime = state.time + Math.max(times.get(p1),
times.get(p2));

                    if (newTime <= 60) {
                        moves.add(new State(newLeft, newRight, "right", newTime));
                    }
                }
            }
        } else {
            for (String p : state.right) {
                Set<String> newLeft = new HashSet<>(state.left);
                newLeft.add(p);

                Set<String> newRight = new HashSet<>(state.right);
```

```
                newRight.remove(p);

                int newTime = state.time + times.get(p);

                if (newTime <= 60) {
                    moves.add(new State(newLeft, newRight, "left", newTime));
                }
            }
        }
        return moves;
    }
}

class State {
    Set<String> left;
    Set<String> right;
    String side;
    int time;

    public State(Set<String> left, Set<String> right, String side, int time) {
        this.left = left;
        this.right = right;
        this.side = side;
        this.time = time;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof State)) return false;
        State s = (State) o;
        return left.equals(s.left) && right.equals(s.right) && side.equals(s.side)
&& time == s.time;
    }

    @Override
    public int hashCode() {
        return Objects.hash(left, right, side, time);
    }
}

public class BridgeBFSExecutor {
```

```
public static List<State> bfs(BridgeCrossingBFS problem) {
    Queue<Pair> queue = new LinkedList<>();
    queue.add(new Pair(problem.initial(), new ArrayList<>()));

    Set<State> visited = new HashSet<>();

    while (!queue.isEmpty()) {
        Pair current = queue.poll();
        State state = current.state;
        List<State> path = current.path;

        if (problem.isGoal(state)) {
            path.add(state);
            return path;
        }

        if (visited.contains(state)) continue;
        visited.add(state);

        for (State nextState : problem.nextMove(state)) {
            List<State> newPath = new ArrayList<>(path);
            newPath.add(state);
            queue.add(new Pair(nextState, newPath));
        }
    }
    return null;
}

public static void main(String[] args) {
    BridgeCrossingBFS bfsProblem = new BridgeCrossingBFS();
    List<State> result = bfs(bfsProblem);

    System.out.println("\nBFS Solution:\n");
    if (result != null) {
        int idx = 1;
        for (State state : result) {
            System.out.println("Step " + idx++ + ": Left=" + state.left +
                ", Right=" + state.right +
                ", Umbrella='" + state.side + "', Time=" + state.time);
        }
    } else {
        System.out.println("No solution found using BFS.");
```

```
        }
    }

    static class Pair {
        State state;
        List<State> path;

        Pair(State state, List<State> path) {
            this.state = state;
            this.path = path;
        }
    }
}
```

BINARY MATRIX USING A STAR

```
import java.util.*;

public class AStarBinaryMatrix {

    static class Node {
        int x, y;
        double g; // cost from start
        double h; // heuristic to goal
        Node parent;

        Node(int x, int y, double g, double h, Node parent) {
            this.x = x;
            this.y = y;
            this.g = g;
            this.h = h;
            this.parent = parent;
        }

        double f() {
            return g + h;
        }
    }

    // 8 directions (including diagonals)
    static final int[][] DIRS = {
```

```
    {1, 0}, {-1, 0}, {0, 1}, {0, -1},
    {1, 1}, {1, -1}, {-1, 1}, {-1, -1}
};

// Euclidean heuristic
static double heuristic(int x1, int y1, int x2, int y2) {
    return Math.hypot(x1 - x2, y1 - y2);
}

static boolean isValid(int x, int y, int[][] grid) {
    int n = grid.length;
    return x >= 0 && y >= 0 && x < n && y < n && grid[x][y] == 0;
}

static List<int[]> reconstructPath(Node end) {
    List<int[]> path = new ArrayList<>();
    Node cur = end;
    while (cur != null) {
        path.add(new int[]{cur.x, cur.y});
        cur = cur.parent;
    }
    Collections.reverse(path);
    return path;
}

static List<int[]> aStar(int[][] grid) {
    int n = grid.length;
    if (n == 0) return Collections.emptyList();
    if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return Collections.emptyList();

    boolean[][] closed = new boolean[n][n];
    // best g cost seen so far
    double[][] bestG = new double[n][n];
    for (double[] row : bestG) Arrays.fill(row, Double.POSITIVE_INFINITY);

    PriorityQueue<Node> open = new PriorityQueue<>(
        (a, b) -> {
            int cmp = Double.compare(a.f(), b.f());
            if (cmp != 0) return cmp;
            return Double.compare(a.h, b.h) < 0 ? -1 : 1;
        }
    );

    Node start = new Node(0, 0, 0.0, heuristic(0, 0, n - 1, n - 1), null);
    bestG[0][0] = 0.0;
```

```
        open.add(start);

        while (!open.isEmpty()) {
            Node cur = open.poll();
            if (closed[cur.x][cur.y]) continue;
            closed[cur.x][cur.y] = true;

            if (cur.x == n - 1 && cur.y == n - 1) {
                return reconstructPath(cur);
            }

            for (int[] d : DIRS) {
                int nx = cur.x + d[0], ny = cur.y + d[1];
                if (!isValid(nx, ny, grid) || closed[nx][ny]) continue;

                double tentativeG = cur.g + 1.0; // uniform step cost; change if you want
diagonal cost sqrt(2)
                if (tentativeG < bestG[nx][ny]) {
                    bestG[nx][ny] = tentativeG;
                    Node next = new Node(nx, ny, tentativeG, heuristic(nx, ny, n - 1, n - 1),
cur);
                    open.add(next);
                }
            }
        }

        return Collections.emptyList(); // no path
    }

    static void printGridWithPath(int[][] grid, List<int[]> path) {
        int n = grid.length;
        char[][] out = new char[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                out[i][j] = grid[i][j] == 1 ? '1' : '0';
            }
        }
        for (int[] p : path) {
            out[p[0]][p[1]] = '*';
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(out[i][j] + " ");
            }
```

```
            System.out.println();
        }
    }

    public static void main(String[] args) {
        // Example binary matrix (0 = free, 1 = blocked)
        int[][] grid = {
            {0, 1, 0, 0, 0},
            {0, 1, 0, 1, 0},
            {0, 0, 0, 1, 0},
            {1, 1, 0, 0, 0},
            {0, 0, 0, 1, 0}
        };

        System.out.println("Input grid (0=free,1=blocked):");
        for (int[] row : grid) System.out.println(Arrays.toString(row));

        List<int[]> path = aStar(grid);
        System.out.println();

        if (path.isEmpty()) {
            System.out.println("No path found from (0,0) to (" + (grid.length-1) + "," +
(grid.length-1) + ").");
        } else {
            System.out.println("Path length: " + path.size());
            System.out.println("Path coordinates: ");
            for (int[] p : path) {
                System.out.print("(" + p[0] + "," + p[1] + ") ");
            }
            System.out.println("\n\nGrid with path (* marks):");
            printGridWithPath(grid, path);
        }
    }
}
```

BINARY MATRIX USING GBFS

```
import java.util.*;

public class GreedyBestFirstSearch {
    static class Cell {
        int x, y;
        double h; // heuristic (Euclidean distance)
```

```
    Cell parent;

    Cell(int x, int y, double h, Cell parent) {
        this.x = x;
        this.y = y;
        this.h = h;
        this.parent = parent;
    }
}

// 8 possible directions (including diagonals)
static int[][] directions = {
    {1, 0}, {-1, 0}, {0, 1}, {0, -1},
    {1, 1}, {1, -1}, {-1, 1}, {-1, -1}
};

// Heuristic (Euclidean Distance)
static double heuristic(int x1, int y1, int x2, int y2) {
    return Math.sqrt((x1 - x2)*(x1 - x2) + (y1 - y2)*(y1 - y2));
}

// Check valid move
static boolean isValid(int x, int y, int[][] grid, boolean[][] visited) {
    int n = grid.length;
    return x >= 0 && y >= 0 && x < n && y < n &&
           grid[x][y] == 0 && !visited[x][y];
}

// Reconstruct path
static List<int[]> constructPath(Cell end) {
    List<int[]> path = new ArrayList<>();
    while (end != null) {
        path.add(new int[]{end.x, end.y});
        end = end.parent;
    }
    Collections.reverse(path);
    return path;
}

// Greedy Best-First Search
static List<int[]> greedyBestFirstSearch(int[][] grid) {
    int n = grid.length;
    boolean[][] visited = new boolean[n][n];

    PriorityQueue<Cell> pq = new PriorityQueue<>(
        (a, b) -> Double.compare(a.h, b.h) // Only prioritize heuristic
    );
```

```
        Cell start = new Cell(0, 0, heuristic(0, 0, n - 1, n - 1), null);
        pq.add(start);

        while (!pq.isEmpty()) {
            Cell curr = pq.poll();

            if (visited[curr.x][curr.y]) continue;
            visited[curr.x][curr.y] = true;

            // Reached goal
            if (curr.x == n - 1 && curr.y == n - 1) {
                return constructPath(curr);
            }

            for (int[] d : directions) {
                int nx = curr.x + d[0], ny = curr.y + d[1];
                if (isValid(nx, ny, grid, visited)) {
                    double hCost = heuristic(nx, ny, n - 1, n - 1);
                    pq.add(new Cell(nx, ny, hCost, curr));
                }
            }
        }
        return Collections.emptyList(); // No path
    }

    public static void main(String[] args) {
        int[][] grid = {
            {0, 0, 1, 0},
            {1, 0, 1, 0},
            {0, 0, 0, 0},
            {1, 1, 0, 0}
        };

        List<int[]> path = greedyBestFirstSearch(grid);

        if (path.isEmpty()) {
            System.out.println("No path found.");
        } else {
            System.out.println("Greedy Best-First Search Path:");
            for (int[] p : path) {
                System.out.print("(" + p[0] + "," + p[1] + ") ");
            }
            System.out.println();
        }
    }
}
```

CHESS EVALUATION

```
import com.github.bhlangonijr.chesslib.*;
import com.github.bhlangonijr.chesslib.move.*;
import com.github.bhlangonijr.chesslib.game.*;

public double evaluate(Board board, boolean
player) {
    // Checkmate
    if (board.isMated()) {
        return player ? 1000 : -1000;
    }

    // Draw conditions
    if (board.isStalemate() ||
board.isInsufficientMaterial() ||
board.isRepetition()) {
        return 0;
    }

    double score = 0.0;

    // Piece values (material evaluation)
    double pawn = 1, knight = 3, bishop = 3.25,
rook = 5, queen = 9;
```

```
    for (Square sq : Square.values()) {
        Piece piece = board.getPiece(sq);
        if (piece != Piece.NONE) {
            double value = 0;
            switch (piece.getPieceType()) {
                case PAWN -> value = pawn;
                case KNIGHT -> value = knight;
                case BISHOP -> value = bishop;
                case ROOK -> value = rook;
                case QUEEN -> value = queen;
            }
            if (piece.getPieceSide() == Side.WHITE)
{
                score += value;
            } else {
                score -= value;
            }
        }
    }

    // Center control
    Square[] centerSquares = {Square.D4,
Square.E4, Square.D5, Square.E5};
    for (Square sq : centerSquares) {
        Piece piece = board.getPiece(sq);
```

```
        if (piece != Piece.NONE) {
            if (piece.getPieceSide() == Side.WHITE)
{
                score += 0.2;
            } else {
                score -= 0.2;
            }
        }
    }

    // Mobility (number of legal moves)
    Board tempBoard = new Board();
    tempBoard.loadFromFen(board.getFen());

    tempBoard.setSideToMove(Side.WHITE);
    int whiteMoves =
tempBoard.legalMoves().size();

    tempBoard.setSideToMove(Side.BLACK);
    int blackMoves =
tempBoard.legalMoves().size();

    score += 0.05 * (whiteMoves - blackMoves);

    return score;
}
```

TICTAC TOE

```
import java.util.Scanner;

public class TicTacToe {

    static char[] board = new char[9];
    static final char HUMAN = 'X';
    static final char AI = 'O';

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Initialize board with empty spaces
        for (int i = 0; i < 9; i++) {
            board[i] = ' ';
        }

        System.out.println("Welcome to Tic Tac
Toe!");
        printBoard();

        while (true) {
            // Human move
            int humanMove;
```

```
        while (true) {
            System.out.print("Enter your move (1-
9): ");
            humanMove = scanner.nextInt() - 1;
            if (humanMove >= 0 && humanMove
< 9 && board[humanMove] == ' ') {
                break;
            }
            System.out.println("Invalid move. Try
again.");
        }
        board[humanMove] = HUMAN;
        printBoard();

        if (isWinner(HUMAN)) {
            System.out.println("You win!");
            break;
        }
        if (isBoardFull()) {
            System.out.println("It's a draw!");
            break;
        }

        // AI move
        System.out.println("Computer's turn:");
        int aiMove = bestMove();
```

```
            board[aiMove] = AI;
            printBoard();

            if (isWinner(AI)) {
                System.out.println("Computer wins!");
                break;
            }
            if (isBoardFull()) {
                System.out.println("It's a draw!");
                break;
            }
        }
        scanner.close();
    }

    static void printBoard() {
        for (int i = 0; i < 3; i++) {
            System.out.println(board[3*i] + " | " +
board[3*i + 1] + " | " + board[3*i + 2]);
            if (i < 2) System.out.println("--+---+--");
        }
        System.out.println();
    }

    static boolean isWinner(char player) {
        int[][] winConditions = {
```

```
        {0,1,2}, {3,4,5}, {6,7,8}, // rows
        {0,3,6}, {1,4,7}, {2,5,8}, // cols
        {0,4,8}, {2,4,6}           // diagonals
    };

    for (int[] condition : winConditions) {
        if (board[condition[0]] == player &&
            board[condition[1]] == player &&
            board[condition[2]] == player) {
            return true;
        }
    }
    return false;
}

static boolean isBoardFull() {
    for (char c : board) {
        if (c == ' ') return false;
    }
    return true;
}

static int minimax(boolean isMaximizing) {
    if (isWinner(AI)) return 1;
    if (isWinner(HUMAN)) return -1;
    if (isBoardFull()) return 0;
```

```
        if (isMaximizing) {
            int bestScore = Integer.MIN_VALUE;
            for (int i = 0; i < 9; i++) {
                if (board[i] == ' ') {
                    board[i] = AI;
                    int score = minimax(false);
                    board[i] = ' ';
                    bestScore = Math.max(score,
bestScore);
                }
            }
            return bestScore;
        } else {
            int bestScore = Integer.MAX_VALUE;
            for (int i = 0; i < 9; i++) {
                if (board[i] == ' ') {
                    board[i] = HUMAN;
                    int score = minimax(true);
                    board[i] = ' ';
                    bestScore = Math.min(score,
bestScore);
                }
            }
            return bestScore;
        }
```

```
    }

    static int bestMove() {
        int bestScore = Integer.MIN_VALUE;
        int move = -1;

        for (int i = 0; i < 9; i++) {
            if (board[i] == ' ') {
                board[i] = AI;
                int score = minimax(false);
                board[i] = ' ';
                if (score > bestScore) {
                    bestScore = score;
                    move = i;
                }
            }
        }
        return move;
    }
}
```

MWGC

```
import java.util.*;
```

```
class State {
    String man, wolf, goat, cabbage;

    State(String m, String w, String g, String c) {
        this.man = m;
        this.wolf = w;
        this.goat = g;
        this.cabbage = c;
    }

    boolean isGoal() {
        return man.equals("R") && wolf.equals("R") && goat.equals("R") && cabbage.equals("R");
    }

    // Check if the state is unsafe
    boolean isValid() {
        // Wolf and goat alone
        if (wolf.equals(goat) && !man.equals(wolf)) return false;
        // Goat and cabbage alone
        if (goat.equals(cabbage) && !man.equals(goat)) return false;
        return true;
    }

    // Generate possible next states
    List<State> getNextStates() {
        List<State> nextStates = new ArrayList<>();
        String newManSide = man.equals("L") ? "R" : "L";

        // 1. Man goes alone
        State s1 = new State(newManSide, wolf, goat, cabbage);
        if (s1.isValid()) nextStates.add(s1);

        // 2. Man takes wolf
        if (man.equals(wolf)) {
            State s2 = new State(newManSide, newManSide, goat, cabbage);
            if (s2.isValid()) nextStates.add(s2);
        }

        // 3. Man takes goat
        if (man.equals(goat)) {
            State s3 = new State(newManSide, wolf, newManSide, cabbage);
            if (s3.isValid()) nextStates.add(s3);
        }

        // 4. Man takes cabbage
        if (man.equals(cabbage)) {
            State s4 = new State(newManSide, wolf, goat, newManSide);
            if (s4.isValid()) nextStates.add(s4);
        }

        return nextStates;
    }

    @Override
    public String toString() {
        return "(" + man + ", " + wolf + ", " + goat + ", " + cabbage + ")";
    }

    @Override
```

```
    public boolean equals(Object obj) {
        if (!(obj instanceof State)) return false;
        State other = (State) obj;
        return man.equals(other.man) && wolf.equals(other.wolf) &&
                goat.equals(other.goat) && cabbage.equals(other.cabbage);
    }

    @Override
    public int hashCode() {
        return Objects.hash(man, wolf, goat, cabbage);
    }
}

public class MWGC {
    public static void main(String[] args) {
        State start = new State("L", "L", "L", "L");
        bfs(start);
    }

    public static void bfs(State start) {
        Queue<List<State>> queue = new LinkedList<>();
        Set<State> visited = new HashSet<>();

        queue.add(new ArrayList<>(Arrays.asList(start)));
        visited.add(start);

        while (!queue.isEmpty()) {
            List<State> path = queue.poll();
            State current = path.get(path.size() - 1);

            if (current.isGoal()) {
                System.out.println("Solution Path:");
                for (State s : path) {
                    System.out.println(s);
                }
                return;
            }

            for (State next : current.getNextStates()) {
                if (!visited.contains(next)) {
                    visited.add(next);
                    List<State> newPath = new ArrayList<>(path);
                    newPath.add(next);
                    queue.add(newPath);
                }
            }
        }
    }
}
```